StSATO — Security Audit Report

Prepared by	infinitetrading.io
Contract	`StSATO`
Source	`contracts/src/stsato.sol`
Deployed	Ethereum Mainnet — `0xdeE7f7A032326148E65EC3068F1c9b29E26B75b3`
Compiler	Solidity `0.8.26`, `optimizer_runs=1_000_000`, `via_ir=true`, `evm_version=shanghai`
OpenZeppelin	v5
Audit Date	May 2026
Test Results	74 / 74 passing

Table of Contents

---

1. Executive Summary

StSATO is a bonding-curve, liquid-staking ERC-20 token backed 1:1 by SATO. It was forked from the **EGGS** contract by EGGS Finance — an audited DeFi protocol implementing the same bonding curve, borrow, leverage, and liquidation mechanics — and adapted for the SATO token on Ethereum mainnet.

The core architecture inherited from EGGS was audited by **Cantina** (report `report-cantinacode-eggs-0130-2` , January 2025). That audit covers the bonding curve invariant, the collateralised lending system, the liquidation loop, and the price monotonicity constraint — all of which are carried over unchanged in StSATO.

Prior to deployment, two contract-level bugs specific to the SATO adaptation were identified and resolved: a critical selector mismatch in the fee burn mechanism, and an uninitialised `lastPrice` variable. Both fixes were independently verified by a full test suite of 74 tests before deployment. Two additional frontend integration pitfalls were identified post-deployment through live transaction monitoring and are documented here with corrective guidance.

The deployed contract is **immutable and ownerless**. `renounceOwnership()` is called atomically inside `setStart()`. No upgrade path, no admin keys, no pause mechanism.

---

2. Lineage and Prior Audit

2.1 EGGS Finance

EGGS Finance is a DeFi protocol that issues asset-backed tokens with bonding-curve price mechanics. The EGGS token is backed by an underlying asset and supports buying, selling, borrowing at up to 99% LTV, leverage looping, and automated liquidations — the exact set of mechanics implemented in StSATO.

StSATO is a direct fork of the EGGS contract, adapted for the SATO ecosystem on Ethereum mainnet. The shared logic includes:

- The bonding curve formula: `price = getBacking() / totalSupply()`
- The `_safetyCheck` price monotonicity invariant
- The `liquidate()` / `drainLiquidations()` day-bucket system
- The full lending lifecycle: `borrow`, `borrowMore`, `repay`, `closePosition`, `flashClosePosition`, `removeCollateral`, `extendLoan`
- The `leverage()` protocol-mint collateral pattern
- The `getMidnightTimestamp` day-rounding helper

2.2 Cantina Audit

The original EGGS contract was audited by **Cantina** prior to its mainnet deployment. Report: `report-cantinacode-eggs-0130-2` (January 30, 2025). The Cantina review covered the contract's core invariants — price monotonicity, collateral solvency, liquidation correctness, and the borrow/repay accounting — and forms the security baseline that StSATO inherits.

All modifications made during the adaptation to SATO are documented in Section 5. Two of those modifications directly resolve issues that would have been critical vulnerabilities in the SATO-specific deployment.

---

3. Scope

File	Description	Status
`contracts/src/stsato.sol`	Full contract (~550 lines)	Reviewed in full

In scope: all contract logic — bonding curve math, lending system, liquidation loop, fee mechanics, bootstrap, ownership, all internal and external functions.

Out of scope: SATO token contract (`0x829f4B62EEBE12Af653b4dD4fFc480966F7d7f09`), frontend, deployment scripts.

---

4. Protocol Architecture

4.1 Bonding Curve

StSATO is an ERC-20 backed 1:1 by SATO. The price is defined as:

$$\text{price} = \frac{\text{getBacking}()}{\text{totalSupply}()}$$

where:

$$\text{getBacking}() = \text{SATO.balanceOf(address(this))} + \text{totalBorrowed}$$

All trade fees accumulate inside the contract as additional SATO backing, causing the price to strictly increase over time. Price is enforced on-chain via `_safetyCheck`, which reverts if `newPrice < lastPrice`.

4.2 Fee Mechanics

Constant	Value	Meaning
`buy_fee`	`990`	Buyer receives `990/1000 = 99%` of the bonding-curve output stSATO
`sell_fee`	`990`	Seller receives `990/1000 = 99%` of bonding-curve output SATO
`buy_fee_leverage`	`10`	1% mint fee on leverage positions
`FEES_BUY` / `FEES_SELL`	`125`	`1/125 = 0.8%` of each trade is routed to `_sendFees`
`_sendFees` split	`1% burn / 99% backing`	Of each fee amount: `feeAmount/100` transferred to `0xdead`; remainder stays in contract

Effective fee to trader: 0.8% per buy/sell. Of that fee, 0.8% × 1% = 0.008% of notional is permanently burned to `0xdead`; the rest increases the SATO backing.

4.3 Lending System

Borrowers lock stSATO collateral and receive SATO at up to 99% LTV. Interest is charged upfront. Loans expire on a midnight timestamp bucket; expired loans are liquidated automatically on the next trade via `liquidate()`. Borrowers may always call `repay()` or `closePosition()` regardless of the liquidation backlog — neither function calls `liquidate()`.

4.4 Key Constants

```
address constant DEAD = 0x000000000000000000000000000000000000dEaD;
uint16 public  constant sell_fee        = 990;
uint16 public  constant buy_fee         = 990;
uint16 public  constant buy_fee_leverage = 10;
uint16 private constant FEE_BASE_1000   = 1000;
uint16 private constant FEES_BUY        = 125;
uint16 private constant FEES_SELL       = 125;
```

---

5. Modifications from EGGS

The following changes were made to the audited EGGS codebase for the StSATO deployment:

5.1 Underlying Token: EGGS → SATO

The backing token was changed from EGGS to SATO (`0x829f4B62EEBE12Af653b4dD4fFc480966F7d7f09` , Ethereum mainnet). The token name and symbol were updated to `stsato` .

5.2 Fee Burn Mechanism — CRITICAL Fix

Original EGGS:

```
IBurnable(eggs).burn(burnAmount);
```

This calls `burn(uint256)` (selector `0x42966c68`) on the backing token.

StSATO:

```
IERC20(sato).safeTransfer(DEAD, burnAmount);
```

Why this was broken in SATO context: The SATO token's burn function has signature `burn(address from, uint256 amount)` and is restricted to the minter role. Calling `IBurnable(sato).burn(burnAmount)` would encode the `uint256` amount as the `address from` parameter and revert on every call with `SafeERC20FailedOperation` . This would have permanently bricked `buy` , `sell` , `borrow` , `leverage` , and all loan operations from the first transaction.

Fix: Replaced with `safeTransfer(DEAD, burnAmount)` . SATO transferred to `0x000...dEaD` is permanently unrecoverable — the economic effect (SATO removed from active circulation) is identical. The fix was identified independently during pre-deployment review and resolved before any mainnet deployment.

5.3 Fair-Launch Bootstrap — All Minted stSATO to `0xdead`

Original EGGS: The bootstrap mint recipient was not necessarily the dead address.

StSATO:

```
function setStart(uint256 satoAmount) external onlyOwner {
    ...
    _mintInternal(address(0x000000000000000000000000000000000000dEaD), satoAmount);
    ...
}
```

All stSATO minted at bootstrap goes permanently to `0xdead` . The deployer seeds the SATO backing but receives **zero stSATO**. This eliminates any deployer price advantage and ensures a starting price of exactly 1 SATO per stSATO for all participants.

5.4 `lastPrice` Initialisation — Bug Fix

Original EGGS: `lastPrice` was left at `0` after `setStart()` . The `_safetyCheck` passes trivially on the first trade (`0 ≤ any price`), providing no invariant protection and leaving an incorrect historical price on-chain.

StSATO:

```
start = true;
lastPrice = 1e18;  // bootstrap price: 1 SATO per stSATO
emit Started(true);
```

`lastPrice` is explicitly set to `1e18` — the exact price at bootstrap — before any trade can occur. The price monotonicity invariant is enforced from the first block.

5.5 Atomic Ownership Renunciation in `setStart()`

`renounceOwnership()` is called at the end of `setStart()` in the same transaction that bootstraps the protocol. After `setStart()`:

- `owner()` returns `address(0)` permanently
- `onlyOwner` functions (`setStart` itself) can never be called again
- No admin of any kind exists

5.6 `lastLiquidationDate` Constructor Initialisation

```
constructor(address _sato) ERC20("stsato", "stsato") Ownable(msg.sender) {
    require(_sato != address(0), "sato cannot be zero address");
    sato = _sato;
    lastLiquidationDate = getMidnightTimestamp(block.timestamp);
}
```

`lastLiquidationDate` is set to `getMidnightTimestamp(block.timestamp)` in the constructor — the next midnight after deployment. This ensures the liquidation loop starts from the deployment epoch rather than from Unix epoch zero (which would require iterating ~20,000 day-buckets on the first trade).

Note on `getMidnightTimestamp` semantics: The function rounds **up** to next midnight, not down:

```
function getMidnightTimestamp(uint256 date) public pure returns (uint256) {
    return (date - (date % 86400)) + 1 days;
}
```

As a consequence, for the first ~24 hours after deployment, `lastLiquidationDate > block.timestamp`. The liquidation while-loop (`while (lastLiquidationDate < block.timestamp)`) simply does not execute during this window — correct behaviour, as no loans existed before deployment. Frontend consumers must guard against `lastLiquidationDate > block.timestamp` in `daysBehind` calculations to avoid BigInt underflow (see Finding 6.4).

5.7 Network

Deployed on **Ethereum mainnet** (chain ID 1) with EVM target `shanghai`. The original EGGS contract targets a different chain.

---

6. Findings

6.1 CRITICAL — IBurnable Selector Mismatch `[RESOLVED before deployment]`

Severity	Critical
Status	Resolved
Location	`_sendFees()`
Introduced by	SATO token's non-standard burn interface

Description: The original EGGS pattern calls `IBurnable(backing).burn(amount)` — encoding `burn(uint256)`. The SATO token exposes `burn(address from, uint256 amount)` with minter-only access. An ABI mismatch would cause every call to `_sendFees()` to revert with `SafeERC20FailedOperation`, making `buy`, `sell`, `borrow`, `borrowMore`, `leverage`, `flashClosePosition`, and `extendLoan` permanently non-functional from the very first transaction.

Impact if unresolved: 100% of trading and borrowing functionality bricked from block 0.

Resolution: Replaced `IBurnable(sato).burn(burnAmount)` with `IERC20(sato).safeTransfer(DEAD, burnAmount)`. Achieves identical economic effect via a different execution path that does not require minter privileges.

Verification: Test `test_buy_feesSentToDead` and `test_fee_1percentBurnedOnFee` confirm SATO accumulates at `0xdead` on every trade.

6.2 HIGH — `lastPrice = 0` After Bootstrap `[RESOLVED before deployment]`

Severity	High
Status	Resolved
Location	`setStart()`
Introduced by	Missing initialisation in EGGS → StSATO adaptation

Description: Without `lastPrice = 1e18` in `setStart()`, the first post-bootstrap trade would call `_safetyCheck` with `lastPrice = 0`. The check `require(lastPrice <= newPrice)` trivially passes, providing no invariant protection on the most critical first transaction. Additionally, `lastPrice()` returning 0 would be misread by any price oracle or frontend as a zero-price condition.

Impact if unresolved: Price invariant unenforced on first trade; incorrect historical price stored on-chain permanently.

Resolution: Added `lastPrice = 1e18` in `setStart()` before `emit Started(true)`.

Verification: Test `test_setStart_setsBackingAndPrice` asserts `lastPrice == 1e18` after `setStart()`.

6.3 LOW — Unbounded `liquidate()` Loop After Extended Dormancy `[Accepted by design]`

Severity	Low (liveness concern, no funds at risk)

Status	Accepted — escape hatch documented
Location	`liquidate()`
Also present in	Original EGGS contract

Description: `liquidate()` iterates from `lastLiquidationDate` to `block.timestamp` in 1-day increments. Each iteration executes two cold SLOADs (~4,400 gas). The gas cost grows linearly with dormancy:

Dormancy	Estimated gas
1 year	~1.6M gas
3 years	~4.8M gas
7 years	~11.2M gas
~14 years	approaches 30M block gas limit

If the protocol is dormant for approximately 14+ years, calls to `buy`, `sell`, `borrow`, `leverage`, and `flashClosePosition` (all of which call `liquidate()` internally) would OOG revert.

Mitigations in place:

1. `drainLiquidations(maxDays)` is permissionless and bounded — any address can call it with e.g. `maxDays=30` to process the backlog iteratively before trading resumes.
2. `repay()` and `closePosition()` do **not** call `liquidate()` — borrowers can always recover collateral regardless of dormancy.
3. `lastLiquidationDate` initialised to deployment epoch prevents any day-0 gas explosion.

Risk in practice: Requires complete protocol inactivity for over a decade. Fully recoverable without any admin action.

6.4 LOW — `isLoanExpired` Returns `true` for Addresses With No Loan `[Accepted — frontend fix required]`

Severity	Low (frontend confusion; no funds at risk on-chain)
Status	Accepted by design (contract); fix applied in frontend documentation
Location	`isLoanExpired()`, `getLoanByAddress()`
Observed in production	tx `0x2ea09d4f7363932eed9c4622231b5d25b908bb35177cb5ff26ded6a8ffcf4352`, block 25153615

Description: `isLoanExpired` is defined as:

```
function isLoanExpired(address _address) public view returns (bool) {
    return Loans[_address].endDate < block.timestamp;
}
```

For an address that has never borrowed, `Loans[addr].endDate = 0`, and `0 < block.timestamp` is always `true`. The function therefore returns `true` for both:

- Addresses with a genuinely expired loan
- Addresses that have never had a loan at all

This ambiguity is intentional at the contract level: `borrow()` uses `isLoanExpired` to delete stale state before opening a new position, which works correctly in all cases. However, frontend implementations that rely on `isLoanExpired` alone to gate loan-related UI can misread the state.

Observed incident (2026-05-22): A user successfully called `borrow(0.205194 SATO, 365 days)` — tx succeeded with status `0x1`, loan recorded on-chain with `collateral = 0.2009 stSATO`, `borrowed = 0.2031 SATO`, `endDate = 2027-05-23`. The frontend displayed "no active loans" because it used `isLoanExpired` as the sole loan-existence check. `isLoanExpired` returns `false` for an active loan, but the frontend may have inverted the logic or used it on the wrong address.

On-chain state was always correct. No funds were at risk.

Required frontend fix:

```
// WRONG — isLoanExpired alone is ambiguous
const hasLoan = !isLoanExpired   // false for both "no loan" and "active loan"

// CORRECT — always check borrowed > 0 first
const [collateral, borrowed, endDate] = await contract.getLoanByAddress(addr)
const hasLoan     = borrowed > 0n
const loanExpired = hasLoan && (endDate < BigInt(Math.floor(Date.now() / 1000)))
const loanActive  = hasLoan && !loanExpired
```

Note: `getLoanByAddress` already returns `(0, 0, 0)` for expired/non-existent loans (it checks `endDate >= block.timestamp` internally), making it the safer single source of truth for UI state.

6.5 INFORMATIONAL — `lastLiquidationDate` Initialised to Future Timestamp Causes Frontend BigInt Underflow `[Frontend fix applied]`

Severity	Informational
Status	Accepted — frontend fix documented
Location	`constructor`, `lastLiquidationDate()`

Description: `getMidnightTimestamp(block.timestamp)` returns the **next** midnight, not the current day's midnight. At the moment of deployment (e.g. 22:08 UTC), `lastLiquidationDate` is set approximately 1h52m into the future. For the first ~24 hours, `lastLiquidationDate > block.timestamp`.

Frontend code that computes `daysBehind = (nowSec - lastLiq) / 86400n` using JavaScript BigInt will produce a massive positive number (BigInt wraps on underflow) when `lastLiq > nowSec`, potentially triggering unnecessary `drainLiquidations()` calls.

Contract behaviour: Correct. The while loop `while (lastLiquidationDate < block.timestamp)` simply does not execute, which is the right behaviour — no loans existed before deployment.

Required frontend fix:

```
const lastLiq    = await contract.lastLiquidationDate()
const nowSec     = BigInt(Math.floor(Date.now() / 1000))
const isCaughtUp = lastLiq >= nowSec
const daysBehind = isCaughtUp ? 0 : Number((nowSec - lastLiq) / 86400n)
```

6.6 INFORMATIONAL — `borrowMore` Duration Calculation Off-By-One Day

Severity	Informational
Status	Accepted — user-favourable
Location	`borrowMore()`
Also present in	Original EGGS contract

Description: In `borrowMore()`, the remaining loan duration is computed as:

```
uint256 todayMidnight  = getMidnightTimestamp(block.timestamp);
uint256 newBorrowLength = (userEndDate - todayMidnight) / 1 days;
```

Since `getMidnightTimestamp` returns the **next** midnight (not the current day's midnight), `todayMidnight` is approximately 24 hours ahead of the current time. This makes `newBorrowLength` approximately 1 day shorter than the true remaining duration, resulting in a slightly lower interest fee than strictly correct.

Impact: The borrower pays slightly less interest when using `borrowMore` (up to 1 day of interest, typically <0.006% of principal). No funds are at risk; the protocol absorbs a negligible fraction of expected revenue.

---

7. Security Properties and Invariant Proofs

7.1 Price Monotonicity

The `_safetyCheck` function is called at the end of every state-changing function and enforces:

```
require(lastPrice <= newPrice, "Price of stsato cannot decrease");
```

where `newPrice = getBacking() × 1e18 / totalSupply()`.

The following table proves the price direction for every operation:

Operation	`getBacking()` change	`totalSupply()` change	Price direction
`buy`	`+satoIn` – tiny SATO burn	`+stsatoOut × 99%`	**strictly ↑**
`sell`	`−satoOut × 99%` – tiny SATO burn	`−stsatoIn`	**strictly ↑**

`borrow`	neutral (SATO out = `totalBorrowed` up) + fee accrual	unchanged	↑
`repay`	neutral (SATO in = `totalBorrowed` down)	unchanged	**= stable**
`closePosition`	neutral	unchanged	**= stable**
`extendLoan`	+fee – tiny burn	unchanged	↑
`leverage`	`+totalFee` – tiny burn	+minted stSATO	↑
`flashClosePosition`	`−toUser` – tiny burn, `−borrowed` (clears debt)	`−collateral`	↑
`liquidate`	`−borrowed` (forgiven debt removed), backing preserved	`−collateral` burned	↑
`removeCollateral`	unchanged	unchanged	**= stable**

Proof sketch for `buy`:

`satoToStsato(X)` uses pre-deposit backing: `minted = X × S / (B − X)`. After the fee:

- Minted stSATO = `X × S × 990 / (B − X) × 1000`
- Net backing increase = `X × (1 − 1/12500)` (0.8% fee, 1% of fee burned)
- New price = `(B + X × 12499/12500) / (S + X × S × 990 / (B − X) × 1000)`

Since `12499/12500 > 990/1000`, the numerator grows proportionally faster than the denominator. Price strictly increases. ✓

7.2 Collateral Solvency

`_safetyCheck` also enforces:

```
require(balanceOf(address(this)) >= totalCollateral, "stsato balance < tracked collateral");
```

This invariant is maintained by construction:

- `borrow` / `borrowMore`: transfers stSATO into contract, increments `totalCollateral` by the same amount
- `closePosition` / `removeCollateral`: transfers stSATO out, decrements `totalCollateral` by the same amount
- `liquidate` / `drainLiquidations`: burns contract-held stSATO, decrements `totalCollateral` by the same amount
- Solidity 0.8+ underflow protection prevents any arithmetic desync

7.3 Supply Floor — `totalSupply` Can Never Reach Zero

`setStart()` mints exactly `satoAmount` stSATO (minimum 1e18) permanently to `0xdead`:

```
_mintInternal(address(0x000000000000000000000000000000000000dEaD), satoAmount);
```

`0xdead` has no private key and cannot call `burn()`. Therefore `totalSupply() ≥ 1e18` in perpetuity. Division by `totalSupply()` in all bonding-curve math is safe.

7.4 Borrower Repayment Always Available

`repay()` and `closePosition()` contain **no** call to `liquidate()`. They are executable in all conditions — regardless of liquidation backlog, gas costs, or protocol dormancy. A borrower can always recover their collateral.

7.5 Overcollateralisation at Liquidation

At borrow time: `collateral = satoToStsatoNoTradeCeil(satoAmount)` — ceiling division ensures `collateral × price ≥ satoAmount`. Debt is `satoAmount × 99/100`. Since price is monotonically non-decreasing, `collateral × currentPrice ≥ satoAmount > debt` at all times. Liquidated positions always clear with a net zero-or-positive effect on the backing ratio.

7.6 No Reentrancy

All public state-changing functions are guarded by `nonReentrant`. State mutations that increment accounting variables precede external `safeTransfer` calls where order allows. The ERC-20 `_burn` call in `sell()` modifies the `msg.sender` balance via internal accounting before any external token movement occurs, and the `nonReentrant` guard prevents any reentrant callback from exploiting an intermediate state.

---

8. Test Coverage

All 74 tests pass (`forge test --match-path "test/StSATO.t.sol"`).

8.1 Bootstrap & Lifecycle

Test	What it verifies
`test_setStart_setsBackingAndPrice`	`lastPrice == 1e18` after bootstrap
`test_setStart_renouncesOwnership`	`owner() == address(0)` after `setStart`
`test_setStart_mintsToDead`	All bootstrap stSATO goes to `0xdead`
`test_setStart_revertsIfAlreadyStarted`	Cannot call `setStart` twice
`test_setStart_revertsIfBelowMinimum`	Minimum 1 SATO required
`test_setStart_revertsIfNotOwner`	`onlyOwner` enforced
`test_start_flagFalseBeforeSetStart`	`start == false` pre-bootstrap

8.2 Buy / Sell

Test	What it verifies
`test_buy_basicMint`	Correct stSATO minted
`test_buy_revertsBeforeStart`	Cannot trade before bootstrap

`test_buy_revertsToZeroAddress`	`receiver != address(0)` enforced
`test_buy_priceIncreasesAfterBuy`	Price strictly increases
`test_buy_feesStayInBacking`	Fee SATO stays in contract
`test_buy_totalMintedIncreases`	Analytics counter increments
`test_buy_receiverDifferentFromCaller`	Receiver can differ from sender
`test_buy_multipleBuyersIncreasePrice`	Multiple buys compound price increase
`test_buy_feesSentToDead`	1% of fee transferred to `0xdead`
`test_buy_totalFeesBurnedTracked`	`totalFeesBurned` counter correct
`test_sell_basicRedeem`	Correct SATO returned on sell
`test_sell_burnReducesTotalSupply`	Supply decreases on sell
`test_sell_getTotalBurnedAccurate`	`getTotalBurned() = totalMinted - totalSupply()`
`test_sell_priceNeverDecreases`	Sell increases price
`test_sell_revertsIfInsufficientBalance`	Cannot sell more than balance

8.3 Borrow / Repay / Close

Test	What it verifies
`test_borrow_basic`	Correct collateral locked, SATO received
`test_borrow_revertsBeforeStart`	Cannot borrow before bootstrap
`test_borrow_revertsZeroAmount`	Zero borrow reverts
`test_borrow_revertsOver365Days`	366+ days reverts
`test_borrow_revertsIfActiveLoanExists`	Cannot double-borrow
`test_borrow_locksCollateral`	stSATO transferred to contract
`test_borrowMore_increasesLoan`	Adds to existing loan
`test_borrowMore_revertsOnExpiredLoan`	Cannot add to expired loan
`test_removeCollateral_returnsExcess`	Excess collateral returned
`test_removeCollateral_revertsIfUndercollateralised`	99% LTV floor enforced
`test_repay_partialReducesBorrow`	`borrowed` decremented correctly
`test_repay_revertsZeroAmount`	Zero repay reverts
`test_repay_revertsFullAmountViaRepay`	Must use `closePosition` for full repay
`test_closePosition_returnsCollateral`	stSATO returned to borrower

`test_closePosition_deletesLoan`	Loan struct deleted
`test_closePosition_revertsOnExpiredLoan`	Cannot close after liquidation
`test_flashClose_returnsNetSATO`	Net SATO returned after flash close
`test_flashClose_revertsOnExpired`	Cannot flash close expired loan
`test_extendLoan_movesEndDate`	`endDate` extended correctly
`test_extendLoan_revertsZeroDays`	Zero extension reverts
`test_extendLoan_revertsIfExpired`	Cannot extend expired loan

8.4 Leverage

Test	What it verifies
`test_leverage_opensPosition`	Position opened correctly
`test_leverage_revertsWithExistingLoan`	Cannot leverage with existing position
`test_leverage_revertsOver365Days`	Duration cap enforced
`test_leverage_canReopenAfterExpiry`	Expired position can be re-opened

8.5 Liquidation

Test	What it verifies
`test_liquidate_processesExpiredLoans`	Expired loans cleared
`test_liquidate_burnsCollateral`	Collateral burned on liquidation
`test_drainLiquidations_processesBoundedChunk`	Bounded loop works correctly
`test_liquidate_idempotentWithNoExpiredLoans`	No-op when nothing to liquidate

8.6 Invariants

Test	What it verifies
`test_invariant_priceNeverDecreases_buyAndSell`	Price monotonicity across mixed activity
`test_invariant_backingEqualsBalancePlusBorrowed`	`getBacking()` accounting identity
`test_invariant_getTotalBurned`	`totalMinted − totalSupply()` accuracy
`test_invariant_priceAfterLiquidation`	Price non-decreasing through liquidation

8.7 Fee Verification

Test	What it verifies
`test_fee_buyFeeIsConstant`	`buy_fee == 990` at compile time

`test_fee_1percentBurnedOnFee`	Exactly 1% of fee amount sent to `0xdead`
`test_fee_99percentStaysInBacking`	99% of fee stays in contract

8.8 Edge Cases & Stress Tests

Test	What it verifies
`test_edge_borrowAndRebuyLoop`	Borrow → sell collateral → repay cycle
`test_edge_safetyCheckRevertOnPriceDecrease`	`_safetyCheck` correctly reverts on manipulated state
`test_edge_isLoanExpiredReturnsFalseForNoLoan`	Zero-loan addresses handled
`test_edge_getLoansExpiringByDate`	Day-bucket query correct
`test_edge_getLoanByAddressReturnsZeroAfterExpiry`	`getLoanByAddress` returns `(0,0,0)` for expired loans
`test_edge_interestFeeCalculation`	Interest formula correct
`test_edge_buyZeroAmount`	Zero buy reverts
`test_edge_multipleUsersFullLifecycle`	Full protocol lifecycle multi-user
`test_stress_buySellCycles50`	50 buy/sell cycles — price monotonicity holds
`test_stress_massiveSingleBuy`	Large buy does not break invariants
`test_stress_sellToBootstrapFloor`	Cannot sell below bootstrap floor
`test_stress_buyLargerThanBootstrap`	Large buy works correctly
`test_stress_extremePriceAppreciation`	Price appreciation at scale
`test_stress_satoSupplyDecreaseMatchesTracker`	SATO balance tracking accurate
`test_stress_totalBurnedAccuracyAtScale`	Burn accounting accurate at scale
`test_stress_tinyBuyAtHighPrice`	Dust buys at high price work

---

9. Access Control

Role	Address	Capabilities
Owner (pre-launch)	Deployer	`setStart()` only
Owner (post-launch)	`address(0)` — permanently renounced	None
Anyone	Public	All trading, borrow, repay, liquidate, drainLiquidations

There are no `pause`, `upgrade`, `rescue`, `setFee`, or `setKeeper` functions. The contract is immutable and fully autonomous after `setStart()`.

---

10. Dependencies

Library	Version	Usage
`ERC20Burnable`	OpenZeppelin v5	Base ERC-20 with `burn()`/`burnFrom()` on stSATO itself (not SATO)
`SafeERC20`	OpenZeppelin v5	All external SATO transfers use `safeTransfer`/`safeTransferFrom`
`Ownable`	OpenZeppelin v5	Owner renounced atomically in `setStart()`
`ReentrancyGuard`	OpenZeppelin v5	Applied to all public write functions
`Math.mulDiv`	OpenZeppelin v5	All fixed-point arithmetic — overflow-safe with optional `Rounding.Ceil`

No external price oracles. No delegate calls. No upgradeable proxy.

---

11. Deployment Record

Item	Value
Contract address	`0xdeE7f7A032326148E65EC3068F1c9b29E26B75b3`
Deploy tx	`0x4cdca94071ecd10e5a3ac07f7a7254e5f0b82a13fc50e3e999798bcf69a24c4d`
Deploy block	25152595
`setStart` tx	`0xe6f0737dc3fc482dc2445498ddc705328c08b2d81169efc59b1a71f27ccb1feb`
`setStart` block	25152640
Bootstrap amount	1e18 SATO (1 SATO)
Etherscan	https://etherscan.io/address/0xdee7f7a032326148e65ec3068f1c9b29e26b75b3

Post-Deployment State Verification

Check	Result
`owner()`	`0x0000000000000000000000000000000000000000` ✓
`start()`	`true` ✓
`lastPrice()`	`1000000000000000000` (1e18) ✓
`getBacking()`	`1e18` SATO ✓

`totalSupply()`	`1e18` stSATO (all held by `0xdead`) ✓
`lastLiquidationDate()`	`1779494400` (2026-05-23 00:00 UTC) ✓
Source verified on Etherscan	✓

---

12. Summary Table

#	Title	Severity	Status
6.1	IBurnable selector mismatch — fee burn bricked	**Critical**	Resolved pre-deployment
6.2	`lastPrice = 0` after bootstrap	**High**	Resolved pre-deployment
6.3	Unbounded `liquidate()` loop after extreme dormancy	**Low**	Accepted — `drainLiquidations` escape hatch
6.4	`isLoanExpired` ambiguous for no-loan addresses	**Low**	Accepted — frontend fix documented
6.5	`lastLiquidationDate` future init causes BigInt underflow in frontend	**Informational**	Frontend fix documented
6.6	`borrowMore` duration off-by-one day	**Informational**	Accepted — user-favourable

No permanent lock of funds is possible. No combination of inputs can permanently prevent a borrower from repaying, a holder from selling, or the protocol from resuming activity.